Teaching Statement

My philosophy of teaching is grounded in a simple principle: students learn computer science most effectively when they are challenged to understand fundamental ideas deeply, apply those ideas to concrete problems, and receive the support necessary to persist when the material becomes difficult. My goal is therefore not simply to transmit technical information, but to help students become independent problem solvers who can reason about unfamiliar problems, communicate their ideas clearly, and apply what they have learned beyond the immediate requirements of a course. I strive to combine theoretical foundations with hands-on experience, connect classroom concepts to real-world applications, and create an environment in which students feel comfortable asking questions and taking intellectual risks.

This philosophy has guided my teaching at Georgia State University, where I have I taught thirteen course offerings, enrolling approximately 800 students, in areas including programming languages, databases, artificial intelligence, algorithms, bioinformatics, and automata. I have also supervised doctoral dissertations, master's theses and projects, undergraduate honors work, and independent studies. These experiences have reinforced my belief that effective teaching extends beyond the classroom to include mentoring, research opportunities, professional development, and support for students' longer-term goals.

Connecting Theory and Practice. One of my central teaching objectives is to help students understand the *why* behind computer science concepts rather than simply learning procedures or programming syntax. Computer science is unusual in that many of its most important ideas are simultaneously mathematical, theoretical, and practical. Students need to understand an algorithm formally, but also know how and when to implement it. My teaching therefore frequently moves between abstraction and implementation.

My course on Programming Language Concepts is particularly representative of this approach. The course combines automata, recursive functions, grammars, and parsing with practical programming in functional and logical languages such as Haskell and Prolog. Students must move between mathematical and computational modes of thinking, making the course intellectually demanding. My objective is not merely for students to complete programming assignments, but to understand the theoretical foundations underlying the languages and paradigms they use. This combination of theory and implementation helps prepare students to approach new languages and unfamiliar computational problems independently.

I use assignments, examples, demonstrations, discussion, and individual assistance to provide multiple opportunities to engage with difficult concepts. In courses involving several programming languages, I have also used common online development environments such as Replit to reduce technical barriers and allow students to focus on the concepts being taught. Because students enter my courses with different backgrounds, I strive to maintain high academic standards while providing the structure and support necessary to meet them. I encourage students to ask questions inside and outside the classroom and provide office hours, appointments, and online meetings when appropriate.

Student evaluations provide evidence that students generally respond positively to this approach. For example, evaluations of my Spring 2023 Artificial Intelligence course included strong ratings for accessibility, clarity, preparation, communication, and achievement of course objectives. Students also rated the course highly for encouraging them to engage with ideas they had not previously considered. Evaluations of Programming Language Concepts have similarly reflected strong student engagement and recognition of the intellectual challenge of the course.

Real-World Applications and Experiential Learning. I believe students learn particularly effectively when they can see how abstract concepts are used to solve problems outside the classroom. I therefore incorporate projects and practical applications whenever appropriate.

My Database Systems course combines relational and entity-relationship models, relational algebra, SQL,

normalization, and database design with a substantial project involving a realistic organizational database. Students apply concepts introduced throughout the course to a larger design problem rather than treating each topic as an isolated exercise. The project culminates in a technical report and oral presentation, requiring students to construct a database and explain their design decisions.

I have taken a similar approach in Artificial Intelligence. I developed the course around foundational concepts including search, heuristic methods, constraint satisfaction, Bayesian reasoning, machine learning, and natural language processing. Students complete projects that apply these techniques to problems of their own choosing. Examples have included an A* search-based snake autopilot game and projects examining chatbots and their technical challenges. These projects encourage students to move beyond isolated algorithms and consider how multiple ideas fit together in a functioning system.

I also encourage students to consider the broader implications of computing. Students in my AI courses have examined systemic bias and responsible AI. One student, Ruqayyah Muse, completed an honors dimension focused on systemic bias in artificial intelligence, conducted a substantial literature review, and investigated approaches for mitigating bias. She subsequently presented her work at Georgia State University's Demo Day and the Georgia State Undergraduate Research Conference. Other students have investigated topics such as microaggressions as adversarial attacks on hate-speech detection systems. Several course projects have been presented at Demo Day, giving students experience communicating their work beyond the classroom.

Teaching Innovation and Pedagogical Scholarship. My commitment to improving student learning has also led me to develop and evaluate pedagogical tools. I believe educational technology is most valuable when it addresses a genuine conceptual difficulty rather than simply adding technology to an existing course just because we can.

One example is my work on an interactive lambda calculus interpreter and visualization tool. Lambda calculus is an important component of Programming Language Concepts, but students can find the nested structure of lambda expressions and the mechanics of beta-reduction difficult to visualize. The tool represents lambda expressions as trees and allows students to interactively perform reductions by selecting nodes. I incorporated the tool into Programming Language Concepts and evaluated its effectiveness as a learning aid. This work resulted in a pedagogical publication and reflects my broader approach: identify a difficult conceptual obstacle, develop a tool or activity that addresses it, and evaluate its effectiveness.

A second example is my work on *parcours*, an open-source tool for teaching computational phylogenetics and ancestral state reconstruction. Developed primarily with undergraduate students and a Biology faculty collaborator, the tool allows students to experiment with concepts associated with the small parsimony problem and Sankoff's algorithm. Its accompanying pedagogical work makes computational phylogenetics accessible to a multidisciplinary audience. The open-source nature of the tool also allows it to be incorporated directly into classroom activities.

Mentoring and Promoting Student Success. I view mentoring as an extension of my teaching responsibilities. Students often need guidance that cannot be provided through lectures or assignments alone, particularly when considering research, graduate school, or careers in computing.

I have supervised students at multiple levels, including doctoral dissertations, master's theses and projects, undergraduate honors work, and independent studies. Three PhD students have completed their dissertations under my supervision so far, and I have served on dissertation and thesis committees for additional students. These experiences have required me to adapt my mentoring to students with different levels of experience and professional goals.

I also make a deliberate effort to help students identify and pursue opportunities beyond their courses. I have written recommendation letters for numerous students applying to graduate programs, internships, fellowships, employment opportunities, and travel awards. I have encouraged students to present their

work at venues such as Demo Day and the Undergraduate Research Conference, providing experience that can be particularly valuable for students who have not previously viewed themselves as researchers or presenters.

My involvement in student success also extends to departmental activities. As a member of the Computer Science Undergraduate Committee's career-readiness subcommittee, I helped refactor career-readiness materials so that career competencies could be integrated into coursework through a simpler web-based format, making them much more accessible to students and faculty.

I have additionally participated in externally funded initiatives supporting student development. Through a Meta AI Learning Alliance Broadening Participation grant, I contributed to efforts to develop educational materials and prepare students for opportunities in artificial intelligence. I have also participated in an NSF-funded Research Experiences for Undergraduates site focused on multidisciplinary graph data analytics, providing undergraduate students with opportunities to conduct research alongside faculty and graduate students. These activities complement classroom teaching by creating pathways from coursework into research and professional development.

Conclusion. My teaching philosophy is centered on helping students become independent and confident problem solvers. I seek to give students a strong theoretical foundation while providing repeated opportunities to apply what they learn. Through projects, interactive tools, research, and professional development, I aim to make computer science both intellectually rigorous and personally meaningful.

Most importantly, I view teaching as a partnership with students. My responsibility is to provide intellectually challenging material, clear structure, meaningful applications, accessibility, and encouragement; students are then challenged to engage actively with the material, persist through difficulties, and take increasing ownership of their learning. The ultimate measure of successful teaching is not simply whether students can reproduce material on an examination, but whether they leave a course with the ability to think computationally, approach unfamiliar problems confidently, communicate technical ideas effectively, and recognize themselves as capable contributors to computer science.